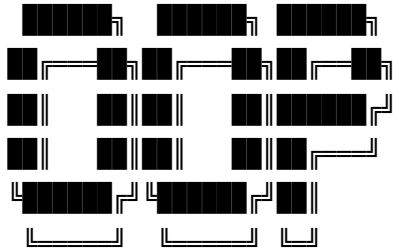




Object-Oriented Programming in C++

Complete Training Guide: Modules 12, 13 & 14



Object-Oriented Programming – C++ Complete Mastery

"OOP is not just a technique — it is a way of thinking about and modeling the real world in code."



Table of Contents

Module	Topic
12.0	Object-Oriented Programming Concepts
12.1	Classes & Objects — Core OOP Principles
13.0	Constructors, Destructors & <code>this</code> Pointer
13.1	Constructors (Default, Parameterized, Copy)
13.2	Destructors
13.3	<code>this</code> Pointer
13.4	Static Variables in C and C++
13.5	Static Members in a Class

Module	Topic
14.0	Friend Functions & Friend Classes
14.1	Friend Functions
14.2	Friend Classes



Prerequisites Checklist

Before starting, ensure you have:

- ☐ Basic C++ syntax (variables, loops, functions)
- ☐ Understanding of pointers and references
- ☐ A C++ compiler (g++, MSVC, or Clang)
- ☐ Completed Module 9 (Data Structures) — recommended

MODULE 12.0 — Object-Oriented Programming Concepts



Learning Objectives

By the end of this module, you will:

- Understand what OOP is and why it was created
- Know the four core pillars of OOP
- Contrast procedural vs object-oriented thinking
- Understand how OOP maps to real-world entities
- Know when to use OOP vs procedural style



Introduction: The Problem OOP Solves

Before OOP existed, programmers wrote **procedural code** — a long sequence of instructions operating on global data. As software grew larger, this became a nightmare:

PROCEDURAL APPROACH (C style) – The Problem:

Global Variables:

```
int studentName[100];  
int studentAge[100];  
float studentGrade[100];
```

Functions scattered everywhere:

```
void printStudent(int id) { ... }  
void updateGrade(int id, float g) { ... }  
void deleteStudent(int id) { ... }
```

PROBLEMS:

- ✗ Data and functions are separate – hard to manage
- ✗ Anyone can accidentally modify global data
- ✗ No natural grouping – code gets messy fast
- ✗ Reusing code for a "Teacher" requires copy-paste

OOP APPROACH (C++ style) – The Solution:

```
class Student {  
    private:  
        string name;    ← Data protected inside  
        int age;  
        float grade;  
    public:  
        void print() { ... }    ← Functions live with data  
        void updateGrade(float g) { grade = g; }  
};
```

BENEFITS:

- ✓ Data and functions bundled together
- ✓ Private data protected from outside access
- ✓ Easy to create 100 Students with "new Student()"
- ✓ Teacher class can INHERIT from Person class

OOP was developed in the 1960s (Simula), popularized in the 1980s (Smalltalk, C++), and today dominates software engineering. Java, Python, C#, Swift, Kotlin — all built around OOP principles.



The Four Pillars of OOP

THE FOUR PILLARS

ENCAPSULATION 	Bundling data + methods Hiding internal details
ABSTRACTION 	Showing only essentials Hiding complexity
INHERITANCE 	Child class reuses parent "is-a" relationship
POLYMORPHISM 	Same interface, many forms Overloading & overriding

MODULE 12.1 — Classes and Objects



Learning Objectives

By the end of this module, you will:

- Define classes and instantiate objects
- Understand the difference between classes and structs
- Apply all four OOP principles with code examples
- Implement access specifiers correctly
- Understand static members and member functions

Classes vs. Structs

Both `class` and `struct` define custom types. Their **only technical difference** in C++ is the **default access specifier**.

CLASS vs STRUCT COMPARISON

struct	class
Default: PUBLIC members	Default: PRIVATE members
<pre>struct Point { int x; // public by default int y; // public by default };</pre>	<pre>class Point { int x; // private by default int y; // private by default };</pre>
<pre>Point p; p.x = 5; //  Works</pre>	<pre>Point p; p.x = 5; //  Error!</pre>
Conventionally used for:	Conventionally used for:
✓ Plain data containers (POD)	✓ Objects with behavior
✓ C compatibility	✓ Full OOP features
✓ Simple groupings	✓ Encapsulated data
Example: Point, Color, Rect	Example: BankAccount, Car, User

```
// struct – Simple data container
struct Point {
    int x; // Public by default
    int y;
};

// class – Encapsulated object with behavior
class BankAccount {
private: // Default for class
    double balance;
    string owner;

public:
    void deposit(double amount) { balance += amount; }
    double getBalance() const { return balance; }
};
```

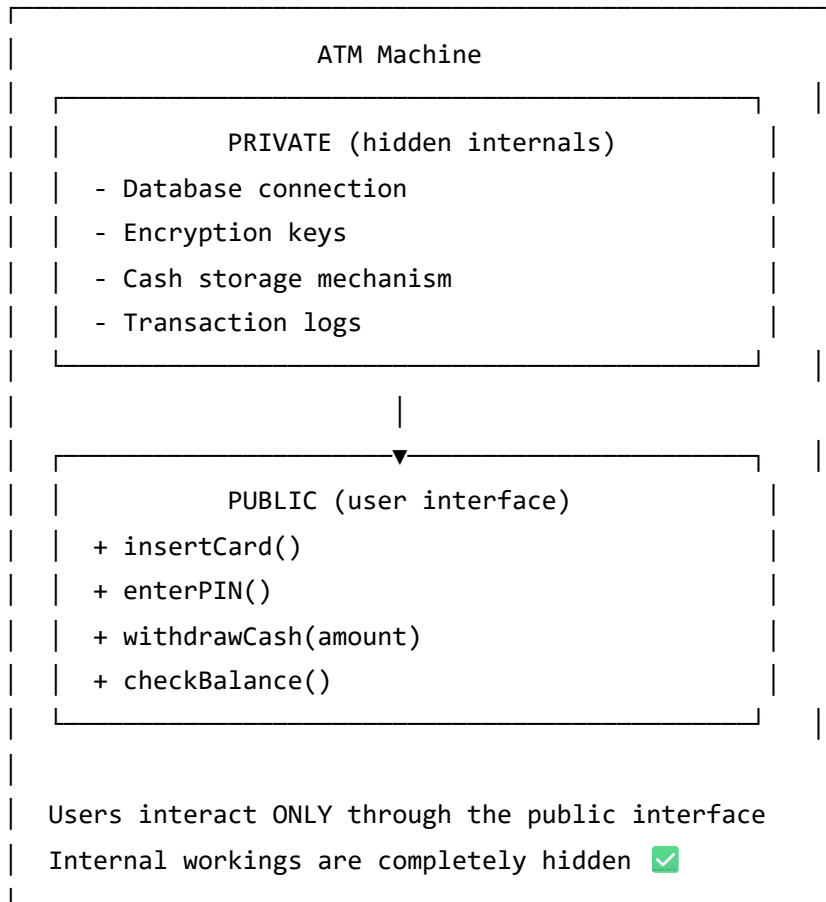
Convention: Use `struct` for plain data, `class` for objects that have behavior and need data protection.



Pillar 1: Data Encapsulation

Encapsulation means bundling data (attributes) and the methods that operate on that data into a single unit (class), and restricting direct access to some components.

ENCAPSULATION ANALOGY: ATM Machine



```

// =====
// ENCAPSULATION EXAMPLE: Bank Account
// =====
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
    // Private data – cannot be accessed directly from outside
    double balance;
    string accountNumber;
    string owner;

    // Private helper method – internal use only
    bool isValidAmount(double amount) const {
        return amount > 0;
    }

public:
    // Constructor
    BankAccount(string name, string accNum, double initial = 0.0)
        : owner(name), accountNumber(accNum), balance(initial) {}

    // Public interface – controlled access to private data
    bool deposit(double amount) {
        if (!isValidAmount(amount)) {
            cout << "Invalid deposit amount!" << endl;
            return false;
        }
        balance += amount;
        cout << "Deposited: $" << amount << endl;
        return true;
    }

    bool withdraw(double amount) {
        if (!isValidAmount(amount)) {
            cout << "Invalid amount!" << endl;
            return false;
        }
        if (amount > balance) {
            cout << "Insufficient funds!" << endl;
            return false;
        }
    }

```



```

    }
    balance -= amount;
    cout << "Withdrew: $" << amount << endl;
    return true;
}

// Getter – read-only access (returns copy, not reference!)
double getBalance() const { return balance; }
string getOwner() const { return owner; }

void printStatement() const {
    cout << "Account: " << accountNumber
         << " | Owner: " << owner
         << " | Balance: $" << balance << endl;
}

};

int main() {
    BankAccount acc("Alice", "ACC-001", 1000.0);

    acc.deposit(500.0);      // Deposited: $500
    acc.withdraw(200.0);     // Withdrew: $200
    acc.withdraw(2000.0);    // Insufficient funds!
    // acc.balance = 9999;   // ❌ ERROR: 'balance' is private

    acc.printStatement();
    // Account: ACC-001 | Owner: Alice | Balance: $1300
    return 0;
}

```

Pillar 2: Abstraction

Abstraction means showing only the **essential features** of an object and hiding the unnecessary implementation details.

ABSTRACTION ANALOGY: Driving a Car

What you SEE (Abstract interface):

 Steering wheel, Pedals, Gear shift
→ `accelerate()`, `brake()`, `steer(angle)`

What you DON'T SEE (Hidden implementation):

 Combustion engine internals
 Fuel injection system
 ECU (Engine Control Unit) code
 Transmission mechanics

You use the car WITHOUT knowing how it works!

```

// =====
// ABSTRACTION: Abstract Shape (using pure virtual functions)
// =====
#include <iostream>
#include <cmath>
using namespace std;

// Abstract base class – defines WHAT shapes can do
// without specifying HOW each shape does it
class Shape {
public:
    // Pure virtual functions – MUST be implemented by subclasses
    virtual double area() const = 0;
    virtual double perimeter() const = 0;
    virtual void describe() const = 0;

    // Virtual destructor (important for base classes!)
    virtual ~Shape() {}

    // Concrete method – same for all shapes
    void printInfo() const {
        cout << "Area: " << area()
             << " | Perimeter: " << perimeter() << endl;
    }
};

// Circle implements the abstract interface
class Circle : public Shape {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}

    double area() const override { return M_PI * radius * radius; }
    double perimeter() const override { return 2 * M_PI * radius; }
    void describe() const override {
        cout << "Circle with radius " << radius;
    }
};

// Rectangle implements the same abstract interface
class Rectangle : public Shape {
private:

```

```

    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}

    double area() const override { return width * height; }
    double perimeter() const override { return 2 * (width + height); }
    void describe() const override {
        cout << "Rectangle " << width << "x" << height;
    }
};

int main() {
    Shape* shapes[] = {
        new Circle(5.0),
        new Rectangle(4.0, 6.0)
    };

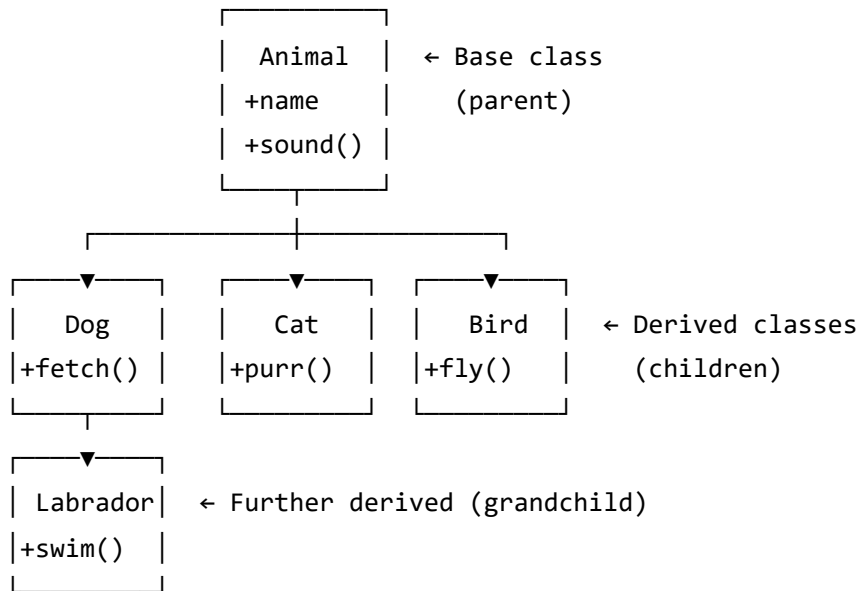
    for (Shape* s : shapes) {
        s->describe();
        cout << " → ";
        s->printInfo(); // Works on ANY shape!
        delete s;
    }
    /* Output:
    * Circle with radius 5 → Area: 78.5398 | Perimeter: 31.4159
    * Rectangle 4x6 → Area: 24 | Perimeter: 20
    */
    return 0;
}

```

Pillar 3: Inheritance

Inheritance allows a class (child/derived) to acquire properties and behaviors of another class (parent/base). It models an **"is-a"** relationship.

INHERITANCE HIERARCHY: Animal Kingdom



"Labrador IS-A Dog IS-A Animal" ✓

```

// =====
// INHERITANCE EXAMPLE: Employee Hierarchy
// =====
#include <iostream>
#include <string>
using namespace std;

// BASE CLASS (Parent)
class Employee {
protected:
    string name;
    int id;
    double baseSalary;

public:
    Employee(string n, int i, double s)
        : name(n), id(i), baseSalary(s) {}

    virtual double calculatePay() const {
        return baseSalary;
    }

    virtual void displayInfo() const {
        cout << "ID: " << id << " | Name: " << name
              << " | Pay: $" << calculatePay() << endl;
    }

    virtual ~Employee() {}
};

// DERIVED CLASS (Child) – inherits Employee
class Manager : public Employee {
private:
    double bonus;

public:
    Manager(string n, int i, double s, double b)
        : Employee(n, i, s), bonus(b) {} // Call parent constructor!

    // OVERRIDE parent's calculatePay
    double calculatePay() const override {
        return baseSalary + bonus; // Manager gets bonus
    }
}

```

```

    void displayInfo() const override {
        cout << "[MANAGER] ";
        Employee::displayInfo(); // Call parent's method
    }
};

// Another derived class
class Contractor : public Employee {
private:
    int hoursWorked;
    double hourlyRate;

public:
    Contractor(string n, int i, int h, double r)
        : Employee(n, i, 0), hoursWorked(h), hourlyRate(r) {}

    double calculatePay() const override {
        return hoursWorked * hourlyRate; // Different pay calculation
    }

    void displayInfo() const override {
        cout << "[CONTRACTOR] ";
        Employee::displayInfo();
    }
};

int main() {
    Employee* staff[] = {
        new Employee("Alice", 1, 3000),
        new Manager("Bob", 2, 5000, 1500),
        new Contractor("Carol", 3, 160, 50)
    };

    cout << "=== Payroll ===" << endl;
    for (Employee* e : staff) {
        e->displayInfo();
        delete e;
    }
    /* Output:
    * ID: 1 | Name: Alice | Pay: $3000
    * [MANAGER] ID: 2 | Name: Bob | Pay: $6500
    * [CONTRACTOR] ID: 3 | Name: Carol | Pay: $8000
    */
}

```

```
*/  
return 0;  
}
```

Types of Inheritance in C++

INHERITANCE TYPES

- | | | |
|-----------------|---|--|
| 1. SINGLE | $A \rightarrow B$ | One parent, one child |
| 2. MULTILEVEL | $A \rightarrow B \rightarrow C$ | Chain of inheritance |
| 3. MULTIPLE | $A + B \rightarrow C$ | C inherits from both A and B
(Can cause Diamond Problem!) |
| 4. HIERARCHICAL | $A \rightarrow B$
$A \rightarrow C$
$A \rightarrow D$ | One parent, many children |
| 5. HYBRID | | Combination of the above |



Pillar 4: Polymorphism

Polymorphism means "many forms" — the same interface behaves differently based on the object type.

POLYMORPHISM: Same call, different behavior

```
shape->draw()
```

```
|  
|  
| └─ if Circle → draws a circle ○  
| └─ if Square → draws a square ◼  
| └─ if Triangle → draws a triangle ▲
```

One interface, many implementations!

Two types of polymorphism:

```

// =====
// 1. COMPILE-TIME POLYMORPHISM: Function Overloading
// Same function name, different parameters
// =====
class Calculator {
public:
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; } // Overloaded
    int add(int a, int b, int c) { return a + b + c; } // Overloaded
    string add(string a, string b) { return a + b; } // Overloaded
};

// Compiler decides which to call based on argument types – at COMPILE time

// =====
// 2. RUNTIME POLYMORPHISM: Virtual Functions + Overriding
// =====
class Animal {
public:
    virtual void speak() const { // 'virtual' enables runtime polymorphism
        cout << "Some generic sound" << endl;
    }
};

class Dog : public Animal {
public:
    void speak() const override { // 'override' is best practice
        cout << "Woof!" << endl;
    }
};

class Cat : public Animal {
public:
    void speak() const override {
        cout << "Meow!" << endl;
    }
};

class Duck : public Animal {
public:
    void speak() const override {
        cout << "Quack!" << endl;
    }
}

```

```
};
```

```
int main() {  
    // RUNTIME polymorphism – decision made at RUNTIME  
    Animal* animals[] = {new Dog(), new Cat(), new Duck()};  
  
    for (Animal* a : animals) {  
        a->Speak(); // Calls the CORRECT version based on actual object type!  
    }  
    // Output:  
    // Woof!  
    // Meow!  
    // Quack!  
  
    for (Animal* a : animals) delete a;  
    return 0;  
}
```

Reusability

Reusability is a core benefit of OOP — write code once, use it many times.

REUSABILITY MECHANISMS

1. INHERITANCE – Reuse parent class code
class Dog : public Animal { ... }
Dog reuses name, sound() from Animal
2. COMPOSITION – Reuse by containing objects
class Car { Engine engine; Wheel wheels[4]; }
Car HAS-A Engine
3. TEMPLATES – Reuse across data types
template<typename T> class Stack { ... }
Works for Stack<int>, Stack<string>, Stack<Dog>
4. LIBRARIES – Reuse pre-built classes
#include <vector> → Already-built dynamic array



Composition vs. Aggregation vs. Inheritance

This is one of the most important design decisions in OOP.

RELATIONSHIP TYPES

INHERITANCE – "IS-A" relationship

Dog IS-A Animal 

```
class Dog : public Animal { ... }
```

Use when: Child is a specialized version of parent

Avoid when: You're just trying to reuse code

COMPOSITION – "HAS-A" (strong ownership)

Car HAS-A Engine (Engine can't exist without Car)

```
class Car {  
    Engine engine; // Engine is PART OF Car  
};  
                // When Car is destroyed, Engine is too
```

Use when: One object is fundamentally part of another

AGGREGATION – "HAS-A" (weak ownership)

University HAS-A Professor (Professor exists independently)

```
class University {  
    Professor* professors[]; // Just a reference/pointer  
};  
                // Professor exists without University
```

Use when: Objects have independent lifecycles

COMPOSITION vs INHERITANCE DECISION

Ask: Is Child a KIND OF Parent?	
YES	NO
↓	↓
Inheritance	Composition
Dog IS-A Animal?	Car IS-A Engine?
YES → inherit	NO → compose

```
// COMPOSITION EXAMPLE
class Engine {
public:
    int horsepower;
    Engine(int hp) : horsepower(hp) {}
    void start() { cout << "Engine started!" << endl; }
};

class Car {
private:
    Engine engine;    // Car OWNS the engine (composition)
    string model;

public:
    Car(string m, int hp) : model(m), engine(hp) {}
    // When Car is created, Engine is created
    // When Car is destroyed, Engine is destroyed

    void drive() {
        engine.start();
        cout << model << " is driving!" << endl;
    }
};
```

Access Specifiers

ACCESS SPECIFIER VISIBILITY TABLE

Specifier	Own Class	Derived Class	Outside Class
private	✓	✗	✗
protected	✓	✓	✗
public	✓	✓	✓

```

class Person {
private:
    string ssn;           // Only accessible in Person

protected:
    string name;          // Accessible in Person AND derived classes
    int age;

public:
    string email;         // Accessible everywhere

    void printInfo() {    // Public method – can access private data
        cout << name << " | " << ssn << endl;
    }
};

class Student : public Person {
public:
    void study() {
        cout << name << " is studying."; // ✅ Can access protected
        // cout << ssn;                    // ❌ Cannot access private
        cout << email << endl;           // ✅ Can access public
    }
};

```

Inheritance Access Modifiers

INHERITANCE ACCESS: How parent members appear in child

In Parent	public inherit	protected inherit	private inherit
public	→ public	→ protected	→ private
protected	→ protected	→ protected	→ private
private	inaccessible	inaccessible	inaccessible

Most common: public inheritance (class Dog : public Animal)

Static Members

Static members belong to the **class itself**, not to any individual object. All objects of the class share one copy.


```

// =====
// STATIC MEMBER EXAMPLE: Counting instances
// =====
class Robot {
private:
    int id;
    static int count; // Shared across ALL Robot objects

public:
    Robot() {
        count++;
        id = count;
        cout << "Robot #" << id << " created." << endl;
    }

    ~Robot() {
        count--;
        cout << "Robot #" << id << " destroyed." << endl;
    }

    static int getCount() { // Static method – no 'this' pointer
        return count;
    }
};

// Define static member OUTSIDE the class
int Robot::count = 0;

int main() {
    cout << "Robots: " << Robot::getCount() << endl; // 0
    Robot r1, r2, r3;
    cout << "Robots: " << Robot::getCount() << endl; // 3
    {
        Robot r4;
        cout << "Robots: " << Robot::getCount() << endl; // 4
    } // r4 destroyed here
    cout << "Robots: " << Robot::getCount() << endl; // 3
    return 0;
}

```



Member Functions

```
class Rectangle {
private:
    double width, height;

public:
    Rectangle(double w, double h) : width(w), height(h) {}

    // Regular member function
    double area() { return width * height; }

    // Const member function – guarantees no modification
    double perimeter() const { return 2 * (width + height); }

    // Inline function – defined inside class body (implicitly inline)
    double getWidth() const { return width; }

    // Function defined outside the class
    void scale(double factor);

    // Operator overloading (also a member function)
    bool operator==(const Rectangle& other) const {
        return width == other.width && height == other.height;
    }
};

// Define outside the class using :: scope resolution
void Rectangle::scale(double factor) {
    width *= factor;
    height *= factor;
}
```



Task 1Z — Practice Problems

Problem 1: Create a `Student` class with:

- Private: `name`, `rollNumber`, `marks[3]`
- Public: Constructor, `calculateAverage()`, `displayInfo()`, `getGrade()`

- Grades: A (≥ 90), B (≥ 75), C (≥ 60), F (< 60)

Problem 2: Create a Shape hierarchy:

- Abstract base: Shape with pure virtual `area()` and `perimeter()`
- Derived: Circle , Rectangle , Triangle
- Write a function that takes any `Shape*` and prints its info

Solution to Problem 1:

```

#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string name;
    int rollNumber;
    double marks[3];

public:
    Student(string n, int roll, double m1, double m2, double m3)
        : name(n), rollNumber(roll) {
        marks[0] = m1; marks[1] = m2; marks[2] = m3;
    }

    double calculateAverage() const {
        return (marks[0] + marks[1] + marks[2]) / 3.0;
    }

    char getGrade() const {
        double avg = calculateAverage();
        if (avg >= 90) return 'A';
        if (avg >= 75) return 'B';
        if (avg >= 60) return 'C';
        return 'F';
    }

    void displayInfo() const {
        cout << "Roll: " << rollNumber << " | " << name
            << " | Avg: " << calculateAverage()
            << " | Grade: " << getGrade() << endl;
    }
};

int main() {
    Student s1("Alice", 101, 92, 88, 95);
    Student s2("Bob", 102, 55, 62, 48);
    s1.displayInfo(); // Roll: 101 | Alice | Avg: 91.67 | Grade: A
    s2.displayInfo(); // Roll: 102 | Bob | Avg: 55 | Grade: F
    return 0;
}

```

Common OOP Mistakes

1. **Making everything public** — defeats the purpose of encapsulation
2. **Overusing inheritance** — "Composition over Inheritance" is a key design principle
3. **Forgetting `virtual destructor`** — memory leak when deleting derived objects via base pointer
4. **God class** — One class that does everything; classes should have a single responsibility
5. **Not using `override`** — Always use `override` keyword to catch typos in virtual function signatures



Quick Reference: OOP Cheat Sheet

CLASS SKELETON:

```
class ClassName {  
private:      // Data members (attributes)  
    type member;  
  
protected:  // For inheritance use  
  
public:      // Interface (methods)  
    ClassName();           // Constructor  
    ~ClassName();          // Destructor  
    type getX() const;     // Getter  
    void setX(type);       // Setter  
    void doSomething();    // Behavior  
};
```

KEY KEYWORDS:

```
virtual    - Enables runtime polymorphism  
override   - Marks overriding functions (C++11)  
abstract   - via pure virtual: virtual fn() = 0;  
static     - Class-level, not instance-level  
const      - Member won't modify object state  
protected - Accessible in class and subclasses
```

MODULE 13.0 — Constructors, Destructors & this Pointer

MODULE 13.1 — Constructors

Learning Objectives

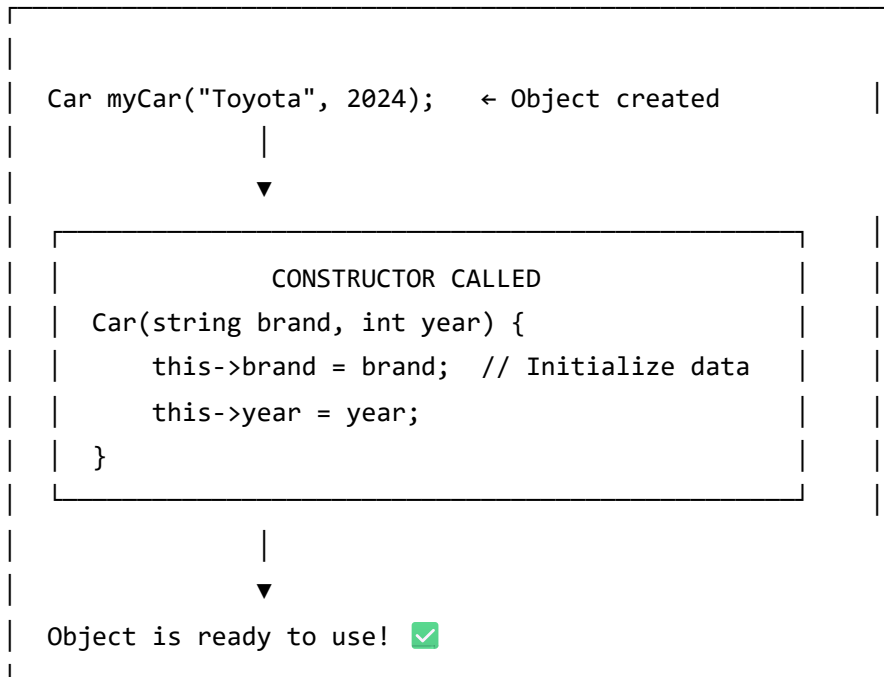
By the end of this module, you will:

- Understand what constructors are and when they're called
- Implement all three constructor types
- Understand the difference between shallow and deep copy
- Know constructor initialization lists
- Avoid common constructor pitfalls

What is a Constructor?

A **constructor** is a special member function that is automatically called when an object is created. It initializes the object's data members.

CONSTRUCTOR LIFECYCLE



Rules for constructors:

- Same name as the class
- No return type (not even `void`)
- Called automatically when object is created
- Can be overloaded (multiple constructors)
- If you define none, compiler provides a default

1 Default Constructor

A **default constructor** takes no parameters (or all parameters have default values).

```

// =====
// DEFAULT CONSTRUCTOR
// =====
#include <iostream>
#include <string>
using namespace std;

class Book {
private:
    string title;
    string author;
    int pages;
    double price;

public:
    // DEFAULT CONSTRUCTOR – no arguments
    Book() {
        title = "Unknown Title";
        author = "Unknown Author";
        pages = 0;
        price = 0.0;
        cout << "Default Book created." << endl;
    }

    // Alternatively, using INITIALIZATION LIST (preferred):
    // Book() : title("Unknown"), author("Unknown"), pages(0), price(0.0) {
    //     cout << "Default Book created." << endl;
    // }

    void display() const {
        cout << "\"" << title << "\" by " << author
             << " | " << pages << " pages | $" << price << endl;
    }

    // Setters to populate default-constructed objects
    void setTitle(string t) { title = t; }
    void setAuthor(string a) { author = a; }
    void setPages(int p) { pages = p; }
    void setPrice(double pr) { price = pr; }
};

int main() {
    Book b1;           // Calls default constructor

```



```
b1.display();           // "Unknown Title" by Unknown Author | 0 pages | $0

Book* b2 = new Book(); // Also calls default constructor
b2->setTitle("1984");
b2->setAuthor("Orwell");
b2->display();
delete b2;

Book shelf[3];          // Calls default constructor 3 times!

return 0;
}
```

2 Parameterized Constructor

A **parameterized constructor** accepts arguments to initialize the object with specific values.

```

// =====
// PARAMETERIZED CONSTRUCTOR
// =====
class Book {
private:
    string title;
    string author;
    int pages;
    double price;

public:
    // PARAMETERIZED CONSTRUCTOR
    Book(string t, string a, int p, double pr)
        : title(t), author(a), pages(p), price(pr) {
        // Initialization list preferred over assignment in body
        // Reason: more efficient, required for const/reference members
        cout << "Book \"" << title << "\" created." << endl;
    }

    // Constructor with DEFAULT parameter values (combines default + parameterized)
    Book(string t = "Unknown", string a = "Unknown", int p = 0, double pr = 0.0)
        : title(t), author(a), pages(p), price(pr) {}

    void display() const {
        cout << "\"" << title << "\" by " << author
            << " | " << pages << " pages | $" << price << endl;
    }
};

int main() {
    // Parameterized – all values specified
    Book b1("The Great Gatsby", "F. Scott Fitzgerald", 180, 12.99);
    b1.display();

    // Using default parameters
    Book b2("Dune");           // Only title specified, rest default
    Book b3;                  // All defaults

    b2.display();
    b3.display();
}

```

```
    return 0;
}
```

Initialization List vs. Assignment

```
class Point {
    const int x; // const member – MUST be initialized in list!
    int& ref;    // reference – MUST be initialized in list!
    int y;

public:
    // INITIALIZATION LIST (correct and preferred):
    Point(int xVal, int& r, int yVal) : x(xVal), ref(r), y(yVal) {}

    // BODY ASSIGNMENT (only works for non-const, non-reference):
    // Point(int xVal, int yVal) {
    //     x = xVal; // ERROR! x is const – cannot assign after creation
    //     y = yVal; // OK
    // }
};
```

Pro Tip: Always prefer initialization lists. They are more efficient (direct initialization instead of default-then-assign) and are required for `const` and reference members.

3 Copy Constructor

A **copy constructor** creates a new object as a copy of an existing object.

COPY CONSTRUCTOR SCENARIOS

When is the copy constructor called?

1. Explicit copy:

```
Student s1("Alice");
```

```
Student s2 = s1;           ← COPY CONSTRUCTOR
```

```
Student s3(s1);           ← COPY CONSTRUCTOR
```

2. Pass by value:

```
void print(Student s);    ← COPY made on entry
```

```
print(s1);
```

3. Return by value:

```
Student create() { Student s; return s; }
```

```
Student s4 = create();    ← COPY made on return
```

Shallow Copy

The **default copy constructor** performs a **shallow copy** — it copies values byte by byte. For simple types this is fine. For **pointer members**, this is dangerous.

```
// =====
// SHALLOW COPY PROBLEM
// =====
class ShallowArray {
public:
    int* data;
    int size;

    ShallowArray(int s) : size(s) {
        data = new int[size];          // Allocate heap memory
        for (int i = 0; i < size; i++) data[i] = i * 10;
    }

    // DEFAULT copy constructor (compiler-generated) performs SHALLOW copy:
    // ShallowArray(const ShallowArray& other) {
    //     size = other.size;
    //     data = other.data; // ⚠️ COPIES THE POINTER, not the data!
    // }

    void print() const {
        for (int i = 0; i < size; i++) cout << data[i] << " ";
        cout << endl;
    }

    ~ShallowArray() {
        delete[] data; // Free heap memory
    }
};

int main() {
    ShallowArray a1(3);    // a1.data → [0, 10, 20] on heap
    ShallowArray a2 = a1;  // Shallow copy: a2.data POINTS TO SAME memory as a1.data!

    a2.data[0] = 999;      // Modify a2

    cout << "a1: "; a1.print(); // a1 is ALSO modified! → 999 10 20 ← BUG!
    cout << "a2: "; a2.print(); // → 999 10 20

    // When program ends:
    // a2's destructor: delete[] data ← frees the memory
    // a1's destructor: delete[] data ← DOUBLE FREE ERROR! Crash!
}
```

```
    return 0;  
}
```

Deep Copy

A **deep copy** creates a completely independent copy — allocates new memory and copies the actual data.

```

// =====
// DEEP COPY – Correct Implementation
// =====
class DeepArray {
public:
    int* data;
    int size;

    // Regular constructor
    DeepArray(int s) : size(s) {
        data = new int[size];
        for (int i = 0; i < size; i++) data[i] = i * 10;
    }

    // DEEP COPY CONSTRUCTOR – manually allocate new memory
    DeepArray(const DeepArray& other) : size(other.size) {
        data = new int[size];          // NEW allocation!
        for (int i = 0; i < size; i++) {
            data[i] = other.data[i];    // Copy VALUES, not pointer
        }
        cout << "Deep copy performed." << endl;

        /*
        * SHALLOW: a2.data —————> [0, 10, 20]
        *           a1.data —————> ^same^
        *
        * DEEP:     a1.data —————> [0, 10, 20]
        *           a2.data —————> [0, 10, 20] ← independent copy!
        */
    }

    // Copy Assignment Operator (Rule of Three: if you define copy ctor, define this too)
    DeepArray& operator=(const DeepArray& other) {
        if (this == &other) return *this; // Self-assignment check

        delete[] data; // Free old memory
        size = other.size;
        data = new int[size]; // Allocate new memory
        for (int i = 0; i < size; i++) {
            data[i] = other.data[i];
        }
        return *this;
    }
}

```

```

void print() const {
    for (int i = 0; i < size; i++) cout << data[i] << " ";
    cout << endl;
}

~DeepArray() {
    delete[] data;
}

};

int main() {
    DeepArray a1(3);          // data → [0, 10, 20]
    DeepArray a2 = a1;        // Deep copy: a2 has its OWN [0, 10, 20]

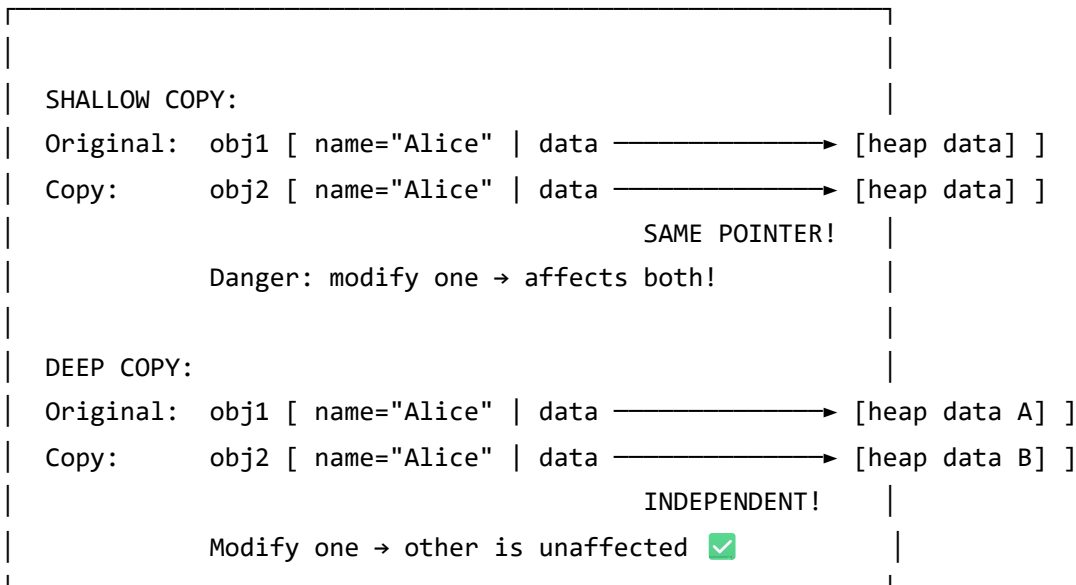
    a2.data[0] = 999;         // Modify a2

    cout << "a1: "; a1.print(); // a1: 0 10 20 ← UNCHANGED ✓
    cout << "a2: "; a2.print(); // a2: 999 10 20

    // No crash! Each has independent memory.
    return 0;
}

```

SHALLOW vs DEEP COPY



The Rule of Three: If you define a **destructor**, **copy constructor**, or **copy assignment operator**, you almost certainly need all three. In C++11+, this extends to the **Rule of Five** (add

move constructor and move assignment operator).

MODULE 13.2 — Destructors

What is a Destructor?

A **destructor** is a special member function called automatically when an object goes out of scope or is explicitly deleted. It performs cleanup — freeing memory, closing files, releasing resources.

DESTRUCTOR LIFECYCLE

```
{
    FileReader reader("data.txt"); ← Constructor
    // ... use reader ...
}
← DESTRUCTOR called automatically when leaving scope

DESTRUCTOR:
~FileReader() {
    fclose(file);    // Close the file
    delete[] buffer; // Free memory
    cout << "FileReader cleaned up!" << endl;
}
```

```

// =====
// DESTRUCTOR – Full Example
// =====
#include <iostream>
using namespace std;

class DatabaseConnection {
private:
    string connectionString;
    bool connected;
    int* queryCache;    // Dynamically allocated resource

public:
    // Constructor – acquire resources
    DatabaseConnection(string connStr) : connectionString(connStr), connected(false) {
        queryCache = new int[100]; // Allocate cache
        connected = true;
        cout << "Connected to: " << connectionString << endl;
    }

    void query(string sql) {
        if (connected)
            cout << "Executing: " << sql << endl;
    }

    // DESTRUCTOR – release resources
    ~DatabaseConnection() {
        if (connected) {
            connected = false;
            cout << "Closing connection to: " << connectionString << endl;
        }
        delete[] queryCache; // Free heap memory
        cout << "DatabaseConnection destroyed." << endl;

        /*
        * Rules for destructors:
        * - Same name as class, prefixed with ~
        * - No parameters, no return type
        * - Cannot be overloaded (only one per class)
        * - Called in REVERSE order of construction for multiple objects
        */
    }
};

```

```

int main() {
    cout << "=== Program Start ===" << endl;

    {
        DatabaseConnection db("mysql://localhost:3306/mydb");
        db.query("SELECT * FROM users");
        // db goes out of scope here ↓
    } // ← DESTRUCTOR called automatically!

    cout << "=== After scope ===" << endl;

    DatabaseConnection* db2 = new DatabaseConnection("postgres://server");
    db2->query("INSERT INTO logs VALUES (...);");
    delete db2; // ← DESTRUCTOR called explicitly

    cout << "=== Program End ===" << endl;
    return 0;
}

/*
 * Output:
 * === Program Start ===
 * Connected to: mysql://localhost:3306/mydb
 * Executing: SELECT * FROM users
 * Closing connection to: mysql://localhost:3306/mydb
 * DatabaseConnection destroyed.
 * === After scope ===
 * Connected to: postgres://server
 * Executing: INSERT INTO logs VALUES (...)
 * Closing connection to: postgres://server
 * DatabaseConnection destroyed.
 * === Program End ===
 */

```

Virtual Destructors — Critical for Inheritance

```
// WITHOUT virtual destructor — MEMORY LEAK!
class Base {
public:
    ~Base() { cout << "Base destroyed" << endl; }
};

class Derived : public Base {
    int* data;
public:
    Derived() { data = new int[100]; }
    ~Derived() { delete[] data; cout << "Derived destroyed" << endl; }
};

Base* obj = new Derived();
delete obj; // ← Only ~Base() called! ~Derived() SKIPPED → MEMORY LEAK!

// WITH virtual destructor — CORRECT
class Base {
public:
    virtual ~Base() { cout << "Base destroyed" << endl; }
};

// Now: delete obj → calls ~Derived() then ~Base() ✅
```

Rule: If a class has virtual functions (i.e., designed to be inherited from), ALWAYS make its destructor virtual.

MODULE 13.3 — this Pointer

What is this ?

The **this pointer** is an implicit pointer available inside every non-static member function. It points to the **current object** — the object that called the method.

THE 'this' POINTER

```
class Box {  
    int width;  
public:  
    void setWidth(int width) {  
        this->width = width; // 'this' resolves name  
    }                        // conflict!  
};  
  
Box b1, b2;  
b1.setWidth(10); → inside: 'this' = &b1 (addr of b1)  
b2.setWidth(20); → inside: 'this' = &b2 (addr of b2)
```

```

// =====
// THIS POINTER – Three Use Cases
// =====
#include <iostream>
using namespace std;

class Chain {
private:
    int value;
    string name;

public:
    Chain(string n, int v) : name(n), value(v) {}

    // USE CASE 1: Resolve naming conflicts
    void setValue(int value) {
        this->value = value; // 'this->value' is the member
        // 'value' alone refers to the parameter!
    }

    // USE CASE 2: Return current object for METHOD CHAINING
    Chain& setName(string name) {
        this->name = name;
        return *this; // Return reference to current object
    }

    Chain& addValue(int v) {
        this->value += v;
        return *this; // Enable chaining
    }

    Chain& print() {
        cout << name << ": " << value << endl;
        return *this;
    }

    // USE CASE 3: Pass current object to another function
    void compare(Chain& other) {
        if (this == &other) {
            cout << "Same object!" << endl;
        } else if (this->value > other.value) {
            cout << name << " is greater." << endl;
        } else {

```

```

        cout << other.name << " is greater." << endl;
    }
}

};

int main() {
    // USE CASE 1: Naming conflict resolution
    Chain c1("Alpha", 10);
    c1.setValue(99);

    // USE CASE 2: Method chaining (fluent interface)
    Chain c2("Beta", 5);
    c2.setName("Gamma").addValue(15).addValue(5).print();
    // ↑ Each method returns *this, enabling dot-chaining
    // Output: Gamma: 25

    // USE CASE 3: Self-comparison
    c1.compare(c2);
    c1.compare(c1); // "Same object!"

    return 0;
}

```

MODULE 13.4 — Static Variables in C and C++

What are Static Variables?

`static` changes the **lifetime** and/or **linkage** of a variable. In C and C++, `static` means different things in different contexts.

STATIC KEYWORD MEANINGS (CONTEXT-DEPENDENT)

1. static LOCAL variable

- Persists across function calls (not destroyed)
- Lives for entire program duration

2. static GLOBAL variable

- Internal linkage (not visible to other files)
- Limits scope to the current .cpp file

3. static CLASS MEMBER

- Shared across ALL instances of the class
- Belongs to class, not to individual objects


```

// =====
// STATIC LOCAL VARIABLE – Persists between calls
// =====
#include <iostream>
using namespace std;

// Counter using static local variable
int generateID() {
    static int nextID = 1; // Initialized ONCE, lives forever
    return nextID++;       // Returns current, then increments

    /*
     * Call 1: nextID = 1, returns 1, nextID becomes 2
     * Call 2: nextID = 2, returns 2, nextID becomes 3
     * ...nextID is NOT reset on each call!
     *
     * WITHOUT static:
     * Call 1: nextID = 1, returns 1 (nextID destroyed on return)
     * Call 2: nextID = 1, returns 1 (always 1!)
     */
}

// Singleton counter
void visitCounter() {
    static int visits = 0;
    visits++;
    cout << "Visit #" << visits << endl;
}

// =====
// STATIC GLOBAL VARIABLE – File-scoped
// =====
static int fileOnlyVariable = 42; // Only accessible in THIS .cpp file
// Other .cpp files cannot see or use 'fileOnlyVariable'

int main() {
    // ID generation
    cout << generateID() << endl; // 1
    cout << generateID() << endl; // 2
    cout << generateID() << endl; // 3

    // Visit counter
    visitCounter(); // Visit #1
}

```

```

    visitCounter(); // Visit #2
    visitCounter(); // Visit #3

    return 0;
}

```

Static vs. Global vs. Local

VARIABLE LIFETIME COMPARISON

Type	Lifetime	Scope	Storage
Local variable	Function run	Function only	Stack
static local	Entire prog	Function only	Data segment
Global variable	Entire prog	Everywhere	Data segment
static global	Entire prog	File only	Data segment

MODULE 13.5 — Static Members in a Class

Static Data Members

A **static data member** is shared by ALL objects of the class. There is only ONE copy, no matter how many objects exist.

REGULAR vs. STATIC MEMBER

Regular member:

obj1 → [id=1 | name="Alice" | instanceData]

obj2 → [id=2 | name="Bob" | instanceData]

obj3 → [id=3 | name="Carol" | instanceData]

Each object has its OWN copy

Static member:

obj1 → [id=1 | name="Alice"]

obj2 → [id=2 | name="Bob"] → count=3

obj3 → [id=3 | name="Carol"]

ALL objects SHARE one copy

```

// =====
// STATIC MEMBERS – Complete Example
// =====
#include <iostream>
#include <string>
using namespace std;

class Player {
private:
    string name;
    int score;
    int id;

    // STATIC data member – shared by all Player objects
    static int playerCount;    // Declaration (no memory allocated yet)
    static int highScore;      // Track all-time high score
    static string topPlayer;

public:
    // Constructor – increments shared counter
    Player(string n, int s = 0) : name(n), score(s) {
        playerCount++;
        id = playerCount;

        // Update high score (shared across all objects)
        if (score > highScore) {
            highScore = score;
            topPlayer = name;
        }
        cout << "Player " << name << " joined! (ID: " << id << ")" << endl;
    }

    // Destructor – decrements shared counter
    ~Player() {
        playerCount--;
        cout << "Player " << name << " left. Remaining: " << playerCount << endl;
    }

    void updateScore(int newScore) {
        score = newScore;
        if (score > highScore) {
            highScore = score;
            topPlayer = name;
        }
    }
};

```

```

    }
}

// STATIC member function – can ONLY access static members
// Has no 'this' pointer (not tied to any object)
static void printLeaderboard() {
    cout << "=== LEADERBOARD ===" << endl;
    cout << "Active Players: " << playerCount << endl;
    cout << "High Score: " << highScore << " by " << topPlayer << endl;
    // cout << name; // ERROR! 'name' is not static – not accessible here
}

static int getPlayerCount() { return playerCount; }

void display() const {
    cout << "[" << id << "]" " " << name << ": " << score << endl;
}
};

// DEFINITION of static members (required outside class!)
int Player::playerCount = 0;
int Player::highScore = 0;
string Player::topPlayer = "None";

int main() {
    cout << "Players: " << Player::getPlayerCount() << endl; // 0

    Player p1("Alice", 500);
    Player p2("Bob", 750);
    Player p3("Carol", 300);

    Player::printLeaderboard();
    // Active Players: 3
    // High Score: 750 by Bob

    p1.updateScore(900);
    Player::printLeaderboard();
    // High Score: 900 by Alice

    {
        Player p4("Dave", 100);
        cout << "Players: " << Player::getPlayerCount() << endl; // 4
    } // p4 destroyed
}

```

```
cout << "Players: " << Player::getPlayerCount() << endl; // 3

return 0;
}
```

Static Member Functions

STATIC MEMBER FUNCTION RULES

- | |
|---|
| <div><div>✓</div><div>CAN access:</div><div>static data members
static member functions
local variables within the function</div></div> <div><div>✗</div><div>CANNOT access:</div><div>non-static data members (no 'this')
non-static member functions
use 'this' keyword</div></div> |
|---|

CALL SYNTAX:

ClassName::staticFunction();	← Via class name
object.staticFunction();	← Via object (allowed)

Task 2 — Practice Problems

Problem 1: Create a `Bank` class that:

- Tracks the total number of accounts (static)
- Tracks total deposits across ALL accounts (static)
- Each account has name, balance, accountID
- Static method: `printBankStats()`

Problem 2: Implement a `Singleton` class using static members (only ONE instance can ever exist).

Solution to Problem 2 (Singleton):

```

class Singleton {
private:
    static Singleton* instance; // The one and only instance
    int data;

    Singleton() : data(0) { // Private constructor!
        cout << "Singleton created." << endl;
    }

    Singleton(const Singleton&) = delete; // No copies!
    Singleton& operator=(const Singleton&) = delete; // No assignment!

public:
    static Singleton* getInstance() {
        if (instance == nullptr) {
            instance = new Singleton(); // Create only if doesn't exist
        }
        return instance; // Always return the same instance
    }

    void setData(int d) { data = d; }
    int getData() const { return data; }
};

Singleton* Singleton::instance = nullptr;

int main() {
    Singleton* s1 = Singleton::getInstance();
    Singleton* s2 = Singleton::getInstance();

    s1->setData(42);
    cout << s2->getData() << endl; // 42 – they're the SAME object!
    cout << (s1 == s2 ? "Same instance" : "Different") << endl; // Same

    return 0;
}

```

✖ Common Mistakes with Constructors/Destructors

1. **Not defining destructor when using `new`** — Memory leak!
2. **Forgetting to define static member outside class** — Linker error
3. **Shallow copy with pointer members** — Data corruption and double-free
4. **Not calling `delete` on `new`** — Memory leak. Use smart pointers (`unique_ptr` , `shared_ptr`) in modern C++
5. **Non-virtual destructor in base class** — Derived destructor never called

MODULE 14.0 — Friend Functions & Friend Classes

MODULE 14.1 — Friend Functions

📌 Learning Objectives

By the end of this module, you will:

- Understand why friend functions exist
- Implement friend functions correctly
- Know the limitations and appropriate use cases
- Understand how friends interact with encapsulation

💛 What is a Friend Function?

A **friend function** is a function that is NOT a member of a class but has access to its **private and protected members**. It's granted special access via the `friend` keyword inside the class definition.

FRIEND FUNCTION CONCEPT

Normal function:

```
void print(Box b) {  
    cout << b.length; // ✗ ERROR: 'length' is private  
}
```

Friend function:

```
class Box {  
    private: int length;  
    friend void print(Box b); // ← Grant access  
};
```

```
void print(Box b) {  
    cout << b.length; // ✓ Allowed! It's a friend  
}
```

ANALOGY: Your house is private. A friend can visit,
a stranger cannot. The 'friend' keyword is the invite.

```

// =====
// FRIEND FUNCTION – Basic Example
// =====
#include <iostream>
using namespace std;

class Temperature {
private:
    double celsius;

public:
    Temperature(double c) : celsius(c) {}

    // Declare friend function INSIDE the class
    friend Temperature add(const Temperature& t1, const Temperature& t2);
    friend void display(const Temperature& t);

    double getCelsius() const { return celsius; }
};

// Define friend function OUTSIDE – NOT part of the class
// Note: No Temperature:: prefix!
Temperature add(const Temperature& t1, const Temperature& t2) {
    // Can access private 'celsius' directly!
    return Temperature(t1.celsius + t2.celsius);
}

void display(const Temperature& t) {
    cout << t.celsius << "°C = "
         << (t.celsius * 9.0/5.0 + 32) << "°F" << endl;
    // Direct access to private member 't.celsius'
}

int main() {
    Temperature t1(25.0); // 25°C
    Temperature t2(10.0); // 10°C

    display(t1); // 25°C = 77°F
    display(t2); // 10°C = 50°F

    Temperature t3 = add(t1, t2);
    display(t3); // 35°C = 95°F
}

```

```
    return 0;  
}
```



Practical Use: Operator Overloading with Friend

The most common real-world use of friend functions is for **operator overloading**, especially the `<<` (stream insertion) and `>>` (stream extraction) operators.

```

// =====
// FRIEND FUNCTION FOR OPERATOR OVERLOADING
// =====
#include <iostream>
#include <string>
using namespace std;

class Vector2D {
private:
    double x, y;

public:
    Vector2D(double x = 0, double y = 0) : x(x), y(y) {}

    // Member operator: Vector + Vector
    Vector2D operator+(const Vector2D& other) const {
        return Vector2D(x + other.x, y + other.y);
    }

    // Member operator: Vector * scalar (vector on LEFT)
    Vector2D operator*(double scalar) const {
        return Vector2D(x * scalar, y * scalar);
    }

    // FRIEND operator: scalar * Vector (scalar on LEFT)
    // Must be friend because left operand is NOT a Vector2D
    friend Vector2D operator*(double scalar, const Vector2D& v) {
        return Vector2D(v.x * scalar, v.y * scalar);
    }

    // FRIEND for << operator (ostream is the left operand)
    friend ostream& operator<<(ostream& out, const Vector2D& v) {
        out << "(" << v.x << ", " << v.y << ")";
        return out; // Allow chaining: cout << v1 << v2
    }

    // FRIEND for >> operator
    friend istream& operator>>(istream& in, Vector2D& v) {
        in >> v.x >> v.y;
        return in;
    }

    double magnitude() const {

```

```

        return sqrt(x*x + y*y);
    }
};

int main() {
    Vector2D v1(3, 4);
    Vector2D v2(1, 2);

    cout << "v1 = " << v1 << endl;           // (3, 4)
    cout << "v2 = " << v2 << endl;           // (1, 2)
    cout << "v1 + v2 = " << v1 + v2 << endl; // (4, 6)
    cout << "v1 * 2 = " << v1 * 2 << endl;   // (6, 8)
    cout << "3 * v1 = " << 3 * v1 << endl;   // (9, 12) ← needs friend!
    cout << "|v1| = " << v1.magnitude() << endl; // 5

    return 0;
}

```



Friend Function Between Two Classes

A friend function can have access to private members of **multiple classes**.

```

// =====
// FRIEND FUNCTION ACCESSING TWO CLASSES
// =====
class Celsius; // Forward declaration

class Fahrenheit {
private:
    double temp;
public:
    Fahrenheit(double t) : temp(t) {}
    friend void convert(const Celsius& c, const Fahrenheit& f);
};

class Celsius {
private:
    double temp;
public:
    Celsius(double t) : temp(t) {}
    friend void convert(const Celsius& c, const Fahrenheit& f);
};

// Single friend function with access to BOTH classes!
void convert(const Celsius& c, const Fahrenheit& f) {
    cout << c.temp << "°C = " << (c.temp * 9.0/5.0 + 32) << "°F" << endl;
    cout << f.temp << "°F = " << ((f.temp - 32) * 5.0/9.0) << "°C" << endl;
}

int main() {
    Celsius c(100);
    Fahrenheit f(212);
    convert(c, f);
    // 100°C = 212°F
    // 212°F = 100°C
    return 0;
}

```



Friend Functions: Pros and Cons

PROS AND CONS

ADVANTAGES	DISADVANTAGES
✓ Operator overloading (<<, >>)	✗ Breaks encapsulation partially
✓ Access private data efficiently	✗ Increases coupling
✓ Symmetric operations (a+b=b+a)	✗ Friend relationship not inherited
✓ Bridge between two classes	✗ Misuse can expose internals

WHEN TO USE:

- ✓ Operator overloading (especially <<, >>)
- ✓ Two closely related classes need to interact
- ✓ Non-member function needs to work with private data

WHEN NOT TO USE:

- ✗ As a shortcut to avoid getters/setters
- ✗ For functions that only need public data
- ✗ When proper encapsulation can solve the problem

MODULE 14.2 — Friend Classes



What is a Friend Class?

A **friend class** is a class all of whose member functions can access the private and protected members of another class.

FRIEND CLASS CONCEPT

```
class Engine {  
    private: int rpm;  
    friend class Car; // Car has access to Engine's  
}; // private members!
```

```
class Car {  
    Engine eng;  
public:  
    void checkRPM() {  
        cout << eng.rpm; // ✅ Allowed!  
    }  
};
```

IMPORTANT:

- Friendship is NOT mutual (Car can access Engine, but Engine CANNOT access Car's private data)
- Friendship is NOT inherited
- Friendship is NOT transitive


```

// =====
// FRIEND CLASS – Full Example: Printer and Document
// =====
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Document {
private:
    string title;
    string content;
    string author;
    bool isDraft;
    vector<string> editHistory;

public:
    Document(string t, string c, string a, bool draft = true)
        : title(t), content(c), author(a), isDraft(draft) {}

    // Declare Printer as friend – all Printer methods can access Document's private data
    friend class Printer;
    friend class DocumentAuditor; // Another friend class

    // Regular public interface
    string getTitle() const { return title; }
    bool getIsDraft() const { return isDraft; }
};

class Printer {
private:
    string printerName;
    int pagesPrinted;

public:
    Printer(string name) : printerName(name), pagesPrinted(0) {}

    // Can access ALL private members of Document!
    void print(const Document& doc) {
        if (doc.isDraft) { // Access private 'isDraft'
            cout << "[DRAFT WATERMARK]" << endl;
        }
    }
}

```

```

    cout << "=== " << printerName << " ===" << endl;
    cout << "Title: " << doc.title << endl;        // Private access
    cout << "Author: " << doc.author << endl;      // Private access
    cout << "Content: " << doc.content << endl;    // Private access
    pagesPrinted++;
    cout << "Total pages printed: " << pagesPrinted << endl;
}

void printSummary(const Document& doc) {
    // Access private members directly
    cout << "DOC[" << doc.title << "]" by " << doc.author;
    cout << (doc.isDraft ? " (DRAFT)" : " (FINAL)") << endl;
}

};

class DocumentAuditor {
public:
    // Audit: access private edit history
    void audit(const Document& doc) {
        cout << "=== AUDIT: " << doc.title << " ===" << endl; // Private!
        if (doc.editHistory.empty()) {
            cout << "No edit history." << endl;
        } else {
            for (auto& entry : doc.editHistory) {
                cout << " - " << entry << endl;
            }
        }
    }
}

};

// This class is NOT a friend – cannot access private members
class EmailSender {
public:
    void sendDoc(const Document& doc) {
        cout << "Sending: " << doc.getTitle() << endl; // Only public access
        // cout << doc.author; // ❌ ERROR: 'author' is private
    }
}

};

int main() {
    Document report("Q3 Report", "Revenue increased by 15%...", "Alice", true);
    Document manual("User Manual", "Welcome to the product...", "Bob", false);

```

```

Printer hp("HP LaserJet");
hp.print(report);
hp.printSummary(manual);

DocumentAuditor auditor;
auditor.audit(report);

EmailSender email;
email.sendDoc(manual);

return 0;
}

```

Friend Relationship Properties

FRIEND RELATIONSHIP RULES

1. NOT SYMMETRIC (One-way):

```
class A { friend class B; };
→ B can access A's private members
→ A CANNOT access B's private members
```

2. NOT TRANSITIVE:

```
A friends B, B friends C
→ C CANNOT access A's private members
(Must explicitly declare C as friend of A)
```

3. NOT INHERITED:

```
class A { friend class B; };
class C : public B {};
→ C is NOT automatically a friend of A
→ Must explicitly declare C as friend
```

4. DECLARED INSIDE the granting class:

```
The class that GRANTS access declares the friend
class A { friend class B; }; ← A grants access
```

Full Comparison: Friend Function vs. Friend Class

Feature	Friend Function	Friend Class
Scope	Single function	All methods of a class
Declaration	<code>friend returnType fn();</code>	<code>friend class ClassName;</code>
Access granted to	One specific function	Entire class
Typical use	Operator overloading	Closely coupled classes
Encapsulation	Minimal violation	Larger violation
Symmetric?	No	No
Inherited?	No	No
Transitive?	No	No

Exercises 14

Exercise 1: Create a `Rectangle` and `Triangle` class. Write a friend function `compareArea()` that compares areas of both shapes.

Exercise 2: Create a `Matrix` class. Overload `<<` and `*` (matrix multiply) using friend functions.

Exercise 3: Implement an `Inventory` and `InventoryManager` friend class. Manager can directly read/write private stock counts.

Solution to Exercise 1:

```

#include <iostream>
using namespace std;

class Triangle; // Forward declaration

class Rectangle {
private:
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}
    double area() const { return width * height; }
    friend void compareArea(const Rectangle& r, const Triangle& t);
};

class Triangle {
private:
    double base, height;
public:
    Triangle(double b, double h) : base(b), height(h) {}
    double area() const { return 0.5 * base * height; }
    friend void compareArea(const Rectangle& r, const Triangle& t);
};

void compareArea(const Rectangle& r, const Triangle& t) {
    double rectArea = r.width * r.height; // Direct private access
    double triArea = 0.5 * t.base * t.height; // Direct private access

    cout << "Rectangle area: " << rectArea << endl;
    cout << "Triangle area: " << triArea << endl;
    cout << (rectArea > triArea ? "Rectangle" : "Triangle") << " is larger." << endl;
}

int main() {
    Rectangle rect(5, 4); // Area = 20
    Triangle tri(6, 8); // Area = 24
    compareArea(rect, tri);
    return 0;
}

```

✗ Common Mistakes with Friends

1. **Overusing friend** — Every time you add a friend, you reduce encapsulation. Consider redesigning with proper getters/setters first.
2. **Assuming friendship is mutual** — If A declares B as friend, B does NOT automatically have access to A.
3. **Thinking friendship is inherited** — Child classes of a friend class are NOT friends.
4. **Using friend instead of interface** — Sometimes an abstract base class with virtual functions is cleaner.



Quick Reference: Friend Cheat Sheet

```
// FRIEND FUNCTION SYNTAX
class MyClass {
    private: int secret;
    friend void accessSecret(MyClass& obj);      // Non-member friend
    friend class OtherClass;                    // Entire class friend
    friend ostream& operator<<(ostream&, const MyClass&); // Stream op
};

// FRIEND FUNCTION DEFINITION (outside, no :: prefix)
void accessSecret(MyClass& obj) {
    cout << obj.secret; // Direct access!
}

// FRIEND OPERATOR DEFINITION
ostream& operator<<(ostream& out, const MyClass& obj) {
    out << obj.secret;
    return out; // Always return ostream& for chaining
}

// USAGE
MyClass obj;
accessSecret(obj);
cout << obj;      // Uses friend << operator
```



5 Capstone Projects — OOP

Project 1: Student Management System

Concepts: Classes, Inheritance, Encapsulation, Static members

- Person base class, Student , Teacher derived
- Static counter for total enrollments
- Report cards with polymorphic generateReport()

Project 2: Banking System

Concepts: Encapsulation, Inheritance, Friend functions

- Account base class, SavingsAccount , CurrentAccount
- Bank friend class that can audit all accounts
- Transaction history with constructors/destructors

Project 3: Vehicle Fleet Manager

Concepts: Composition, Inheritance, Polymorphism

- Engine , Wheel composed into Vehicle
- Car , Truck , Motorcycle inherit from Vehicle
- Virtual maintenanceCost() for polymorphic billing

Project 4: Game Character System

Concepts: All four OOP pillars, Copy constructors, Deep copy

- Character base with Warrior , Mage , Archer
- Deep copy for character cloning
- Inventory class with overloaded << and + operators

Project 5: Library Management System

Concepts: Static members, Friend classes, Constructors

- Book , Member , Library (friend class)
- Library can access private Book availability directly
- Static total books count, Destructor cleanup



Interview Questions — OOP

1. What is the difference between `class` and `struct` in C++?
2. What are the four pillars of OOP? Explain each.
3. What is the difference between shallow and deep copy?
4. Why do we need a virtual destructor?
5. What is the `this` pointer? When would you use it?
6. What is the Rule of Three (Rule of Five)?
7. What is the difference between composition and inheritance? When to use each?
8. Can a constructor be virtual? Can a destructor? Why?
9. What are friend functions? When are they appropriate?
10. What is static binding vs. dynamic binding?
11. What is the diamond problem in multiple inheritance?
12. What is a copy constructor? When is it implicitly called?
13. What does `friend class X` mean? Is friendship symmetric?
14. What is the difference between method overloading and overriding?
15. Can static member functions be virtual?



Glossary — OOP Terms

Term	Definition
Class	Blueprint for creating objects; defines attributes and behaviors
Object	Instance of a class; actual entity in memory

Term	Definition
Encapsulation	Bundling data and methods; hiding internal details
Abstraction	Exposing only essential features; hiding implementation
Inheritance	Child class acquiring properties from parent class
Polymorphism	Same interface with different behaviors
Constructor	Special function called when object is created
Destructor	Special function called when object is destroyed
this pointer	Implicit pointer to the current object inside member functions
Static member	Shared by all instances of a class
Friend function	Non-member function with access to private/protected members
Friend class	All methods of a class have access to another class's private members
Shallow copy	Copies values including pointers (dangerous with dynamic memory)
Deep copy	Copies values AND allocates new memory for pointer members
Virtual function	Enables runtime polymorphism; overridden in derived classes
Pure virtual	<code>= 0</code> makes class abstract; must be implemented by derived classes
Access specifier	<code>public</code> , <code>protected</code> , <code>private</code> — controls member visibility
Composition	HAS-A: object contains another object (strong ownership)
Aggregation	HAS-A: object references another (weak ownership)
Initialization list	<code>: member(value)</code> — initializes before constructor body runs
Rule of Three	If you define destructor/copy ctor/copy assign, define all three
Scope resolution	<code>::</code> operator — accesses class or namespace members



Additional Resources

Books

- **"The C++ Programming Language"** — Bjarne Stroustrup (C++ creator)
- **"Effective C++"** — Scott Meyers — 55 specific tips for better C++
- **"Design Patterns"** — Gang of Four — Classic OOP design patterns
- **"Clean Code"** — Robert Martin — Writing maintainable OOP code

Online Resources

- cppreference.com — Authoritative C++ reference
- learncpp.com — Excellent free C++ tutorials
- isocpp.org — Official ISO C++ website
- cplusplus.com/doc/tutorial — C++ tutorial with OOP section

Practice

- **LeetCode OOP problems** — Search "design" tagged problems
- **HackerRank C++ domain** — OOP-specific challenges



Next Steps After Completion

ADVANCED OOP TOPICS TO EXPLORE NEXT

IMMEDIATELY AFTER:

- └─ Templates (Generic Programming)
 - └─ `class Stack<T>, function<T> max(T, T)`
- └─ Exception Handling
 - └─ `try/catch/throw`
- └─ STL Containers (`vector`, `map`, `set`, `list`)

INTERMEDIATE:

- └─ Smart Pointers (`unique_ptr`, `shared_ptr`)
- └─ Move Semantics (C++11 rvalue references)
- └─ Lambda Functions
- └─ Design Patterns (Singleton, Factory, Observer)

ADVANCED:

- └─ CRTP (Curiously Recurring Template Pattern)
- └─ Variadic Templates
- └─ Metaprogramming
- └─ Concepts (C++20)